AI BASED PERSONAL FINANCE MANAGER

Block Diagrams (all)

Department of Computer Science and Engineering, TKR College of Engineering & Technology

Batch C11 (CSE-C) · Academic year [ACADEMIC YEAR]

Table 1: Project team

S.No.	Student name	Roll number
1	Mekala Kalyan	23K91A05F0
2	Rajanala Sai Hemanth	23K91A05K3
3	Middala Surya Teja Reddy	23K91A05F2
4	Manga Akash	23K91A05E5

Project guide: N Padmavathi

Chapter and section numbers quoted in the text (for example Chapter 8 or Section 6.2) refer to the full project report, 17.Final Document. Figures and tables in this document are numbered from 1.

1. INDEX OF DIAGRAMS

Every diagram was generated from the real design by `scripts/docs/make_diagrams.py` (matplotlib, 150 dpi) and mirrors the code in `database/models.py`, `backend/main.py`, `services/`, `ml/` and `auth/authentication.py`. The entity-relationship diagram shows the six tables of the SQLite database exactly as the SQLAlchemy models define them.

Table 2: Diagrams in this document

No.	Diagram	Image file
1	System architecture	`docs/diagrams/01_system_architecture.png`
2	High-level workflow	`docs/diagrams/02_high_level_workflow.png`
3	Data flow diagram, level 0	`docs/diagrams/03_dfd_level0.png`
4	Data flow diagram, level 1	`docs/diagrams/04_dfd_level1.png`
5	Data flow diagram, level 2 (process 4.0)	`docs/diagrams/05_dfd_level2.png`
6	UML class diagram	`docs/diagrams/06_uml_class.png`
7	Sequence diagram of the main flow	`docs/diagrams/07_sequence_diagram.png`
8	Use-case diagram	`docs/diagrams/08_use_case.png`
9	Entity-relationship diagram	`docs/diagrams/09_er_diagram.png`
10	Flowchart of the core algorithm	`docs/diagrams/10_flowchart_algorithm.png`
11	Expense categoriser pipeline	`docs/diagrams/11_categoriser_pipeline.png`

No.	Diagram	Image file
12	Financial health score model	`docs/diagrams/12_health_score_model.png`

2. DIAGRAMS

System architecture

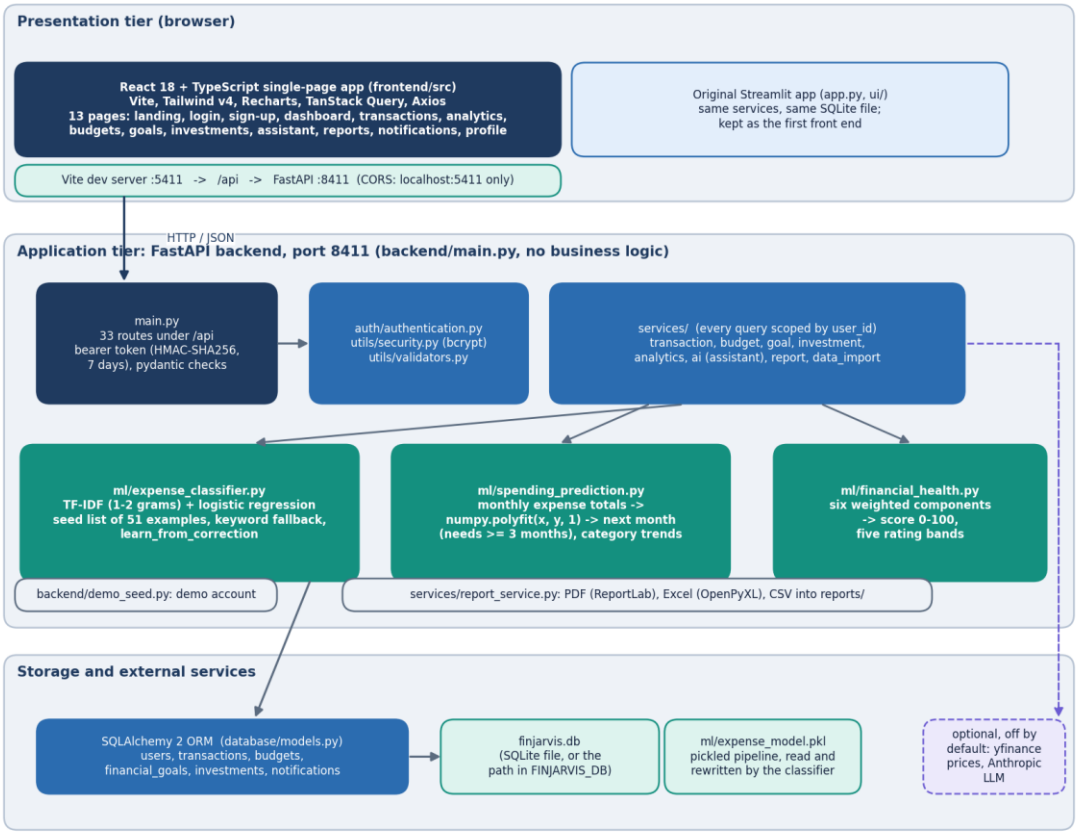


Figure 1: Layered architecture of FINJARVIS: React single-page app, FastAPI layer with 33 routes, services and ML modules, SQLAlchemy ORM and the SQLite file, with the optional price and LLM services shown dashed.

The presentation tier is a React 18 and TypeScript single-page app (Vite, Tailwind v4, Recharts, TanStack Query, Axios) with 13 pages, served on port 5411 and proxying every /api call to the FastAPI application on port 8411. The original Streamlit app (app.py and ui/) still exists and calls the same service functions over the same SQLite file. backend/main.py holds 33 routes and no business logic: it validates input with pydantic, reads the user id from an HMAC-SHA256 bearer token (7-day expiry) and calls the services. The services (transaction, budget, goal, investment, analytics, assistant, report, import) scope every query by user_id, and three ML modules sit beside them: the TF-IDF and logistic-regression categoriser, the linear-trend forecaster built on numpy.polyfit, and the six-component health score. SQLAlchemy 2 maps six tables to finjarvis.db, the categoriser's fitted pipeline is pickled to ml/expense_model.pkl, and the yfinance price refresh and the Anthropic LLM layer are optional and off by default.

High-level workflow

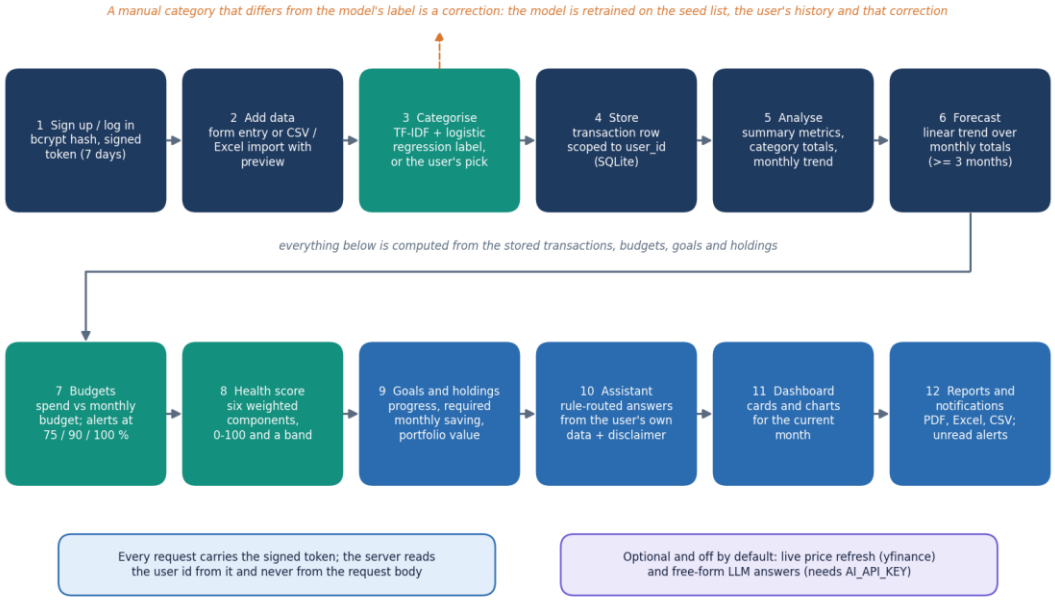


Figure 2: End-to-end workflow: sign-up, data entry or import, categorisation, storage, analysis, forecast, budgets, health score, goals, assistant, dashboard and reports.

A user signs up or logs in (bcrypt-hashed password, signed token) and adds transactions by form or by importing a CSV or Excel file with a preview. Each description is labelled by the TF-IDF and logistic-regression model, or by the category the user picked, and the row is stored under the user's id. A manual pick that differs from the model's label is treated as a correction and retrains the model on the seed list, the user's history and that pick. From the stored rows the analytics layer computes summary metrics, category totals and the monthly trend, and the forecaster fits a straight line to the monthly expense totals when at least three months exist. Budgets compare the month's spending with each limit and raise alerts at 75, 90 and 100 percent, the health score combines six weighted components, and goals and holdings report their progress and value. The assistant answers questions from the same data, the dashboard shows the month at a glance and reports can be downloaded as PDF, Excel or CSV.

Data flow diagram, level 0

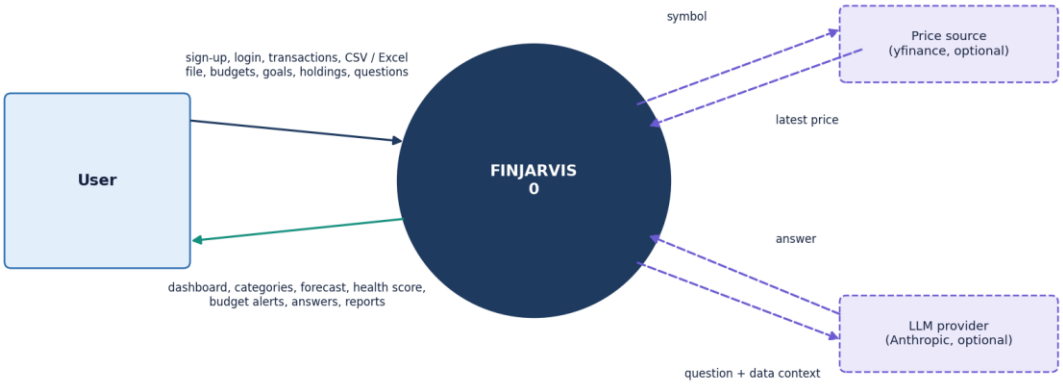


Figure 3: Context diagram (DFD level 0): FINJARVIS as one process with the user, the optional price source and the optional LLM provider as external entities.

At level 0 the whole product is one process. The user sends sign-up and login details, transactions, CSV or Excel files, budgets, goals, holdings and questions, and receives the dashboard, categories, the forecast, the health score, budget alerts, answers and reports. The price source (yfinance) receives a ticker symbol and returns the latest price when a holding is refreshed; this needs internet access and was not exercised in the measurements. The LLM provider receives a question and a summary of the user's data and returns free text, but only when AI_API_KEY is set; the rule-based answers do not leave the machine. No bank, SMS or payment system is connected.

Data flow diagram, level 1

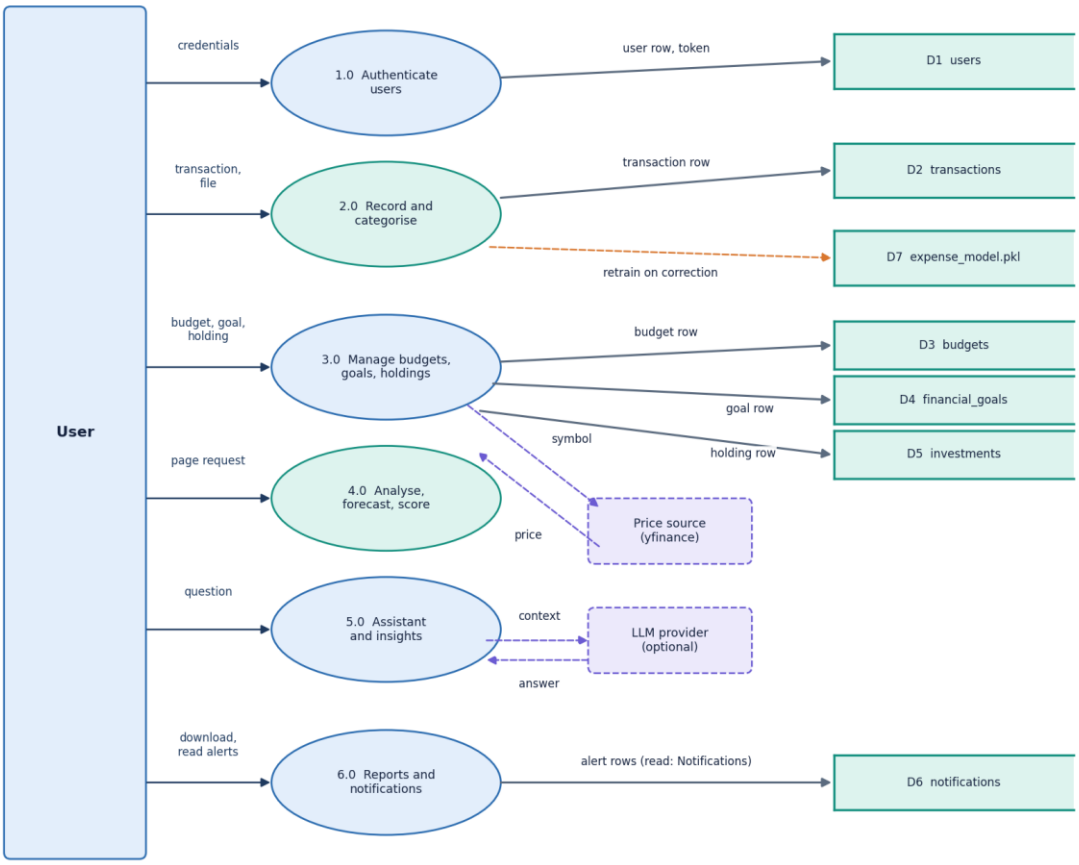


Figure 4: DFD level 1: six processes (authenticate, record and categorise, manage budgets, goals and holdings, analyse and forecast, assistant and insights, reports and notifications) and seven data stores.

Process 1.0 checks credentials against D1 (users) and returns a token. Process 2.0 predicts a category with the pickled model (D7), stores the transaction in D2 and, on a correction, rewrites D7; it also handles file import. Process 3.0 writes budgets (D3), goals (D4) and holdings (D5) and, for a holding, may ask the price source for a latest price. Process 4.0 reads D2 to D5 to build the dashboard, the analytics page, the forecast and the health score, and it raises budget alerts into D6 (notifications). Process 5.0 answers a question from the same stores and may send a context summary to the optional LLM provider, and process 6.0 produces PDF, Excel and CSV reports and lists and marks notifications. Every process filters its reads and writes by the user id taken from the token.

Data flow diagram, level 2 (process 4.0)

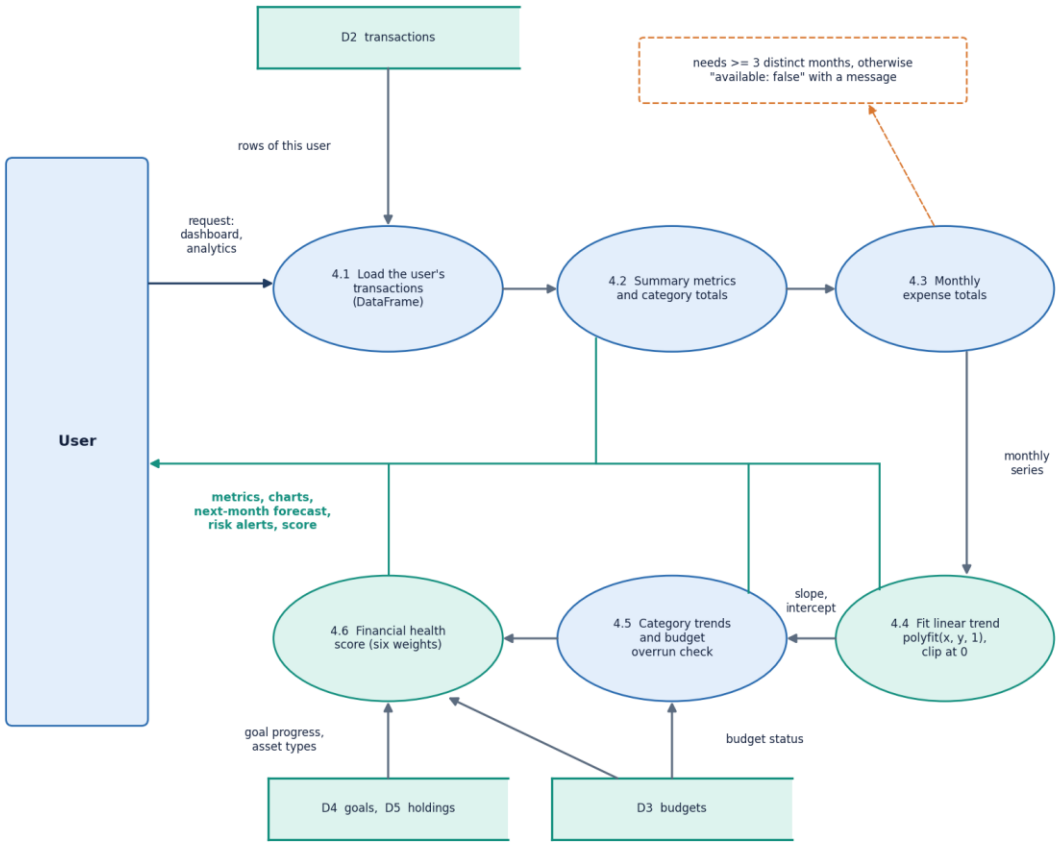


Figure 5: DFD level 2 of process 4.0: load the user's rows, compute metrics and category totals, build monthly expense totals, fit the linear trend, forecast categories and check budget overruns, and score financial health.

Process 4.0 starts by loading the signed-in user's transactions into a data frame (4.1) and computing the summary metrics and category totals (4.2). The monthly expense totals (4.3) feed the forecaster (4.4), which fits a least-squares line with numpy.polyfit(x, y, 1), clips the next-month value at zero and labels the trend increasing, decreasing or stable using a slope threshold of one rupee per month; with fewer than three months it answers that no prediction is available. Category trends are fitted the same way and compared with this month's budgets to list predicted overruns (4.5). The health score (4.6) combines the metrics with the budget status, goal progress and the asset types held. The results travel back to the user as JSON for the dashboard and analytics pages.

UML class diagram

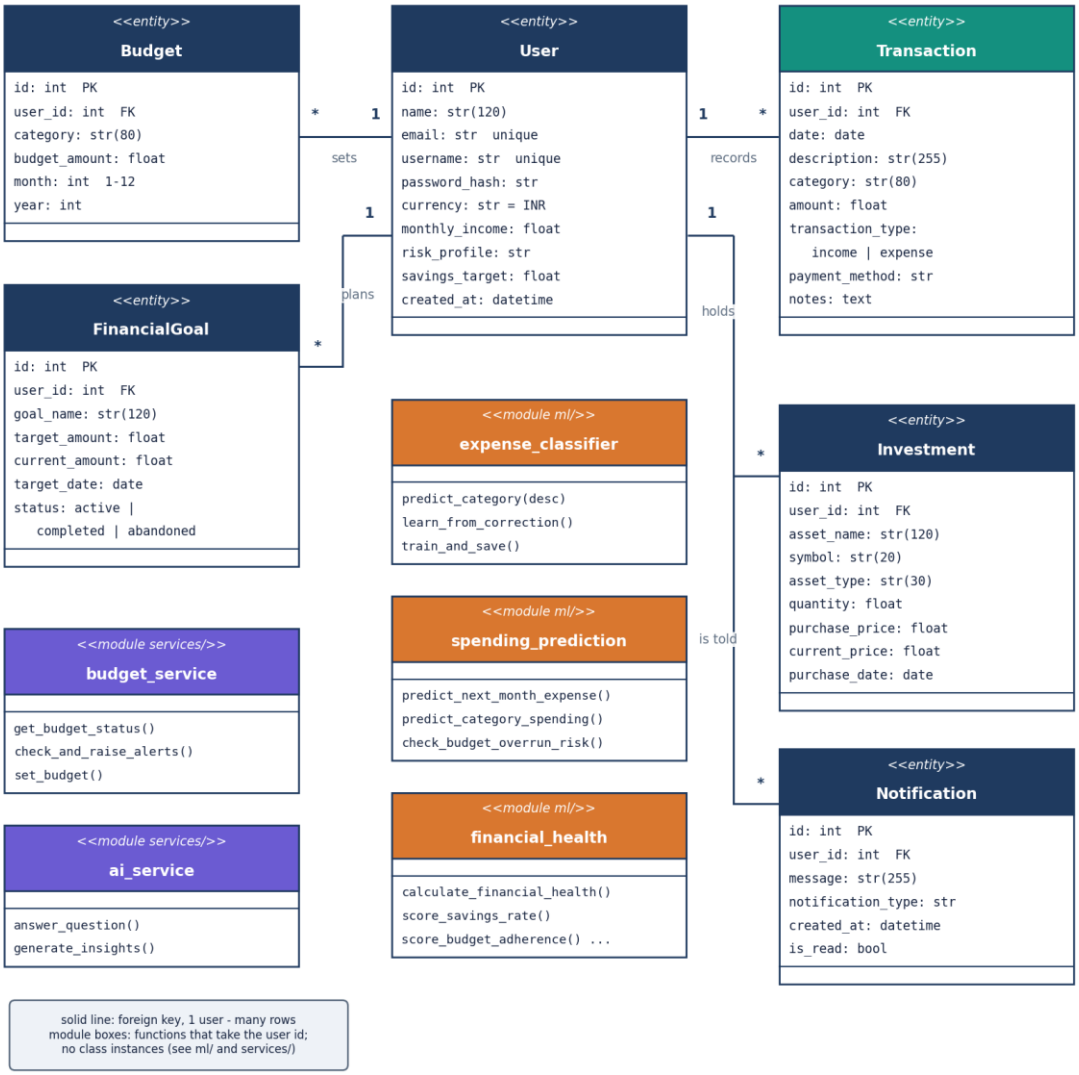


Figure 6: UML class diagram: the six ORM entities (User with one-to-many links to Budget, FinancialGoal, Transaction, Investment and Notification) and the function modules of ml/ and services/.

The entity classes are exactly the SQLAlchemy models in database/models.py: User owns many Transactions, Budgets, FinancialGoals, Investments and Notifications through a user_id foreign key, and each relation uses cascade all, delete-orphan. The orange boxes are the ML modules (the categoriser with predict_category and learn_from_correction, the forecaster with predict_next_month_expense, predict_category_spending and check_budget_overrun_risk, and calculate_financial_health with its six score functions). The purple boxes are two of the service modules, budget_service (status per category and the alert check) and ai_service (rule-routed answers and insights). These modules are

plain functions that take the user id and open their own database session, so they appear as module boxes rather than classes with state.

Sequence diagram of the main flow

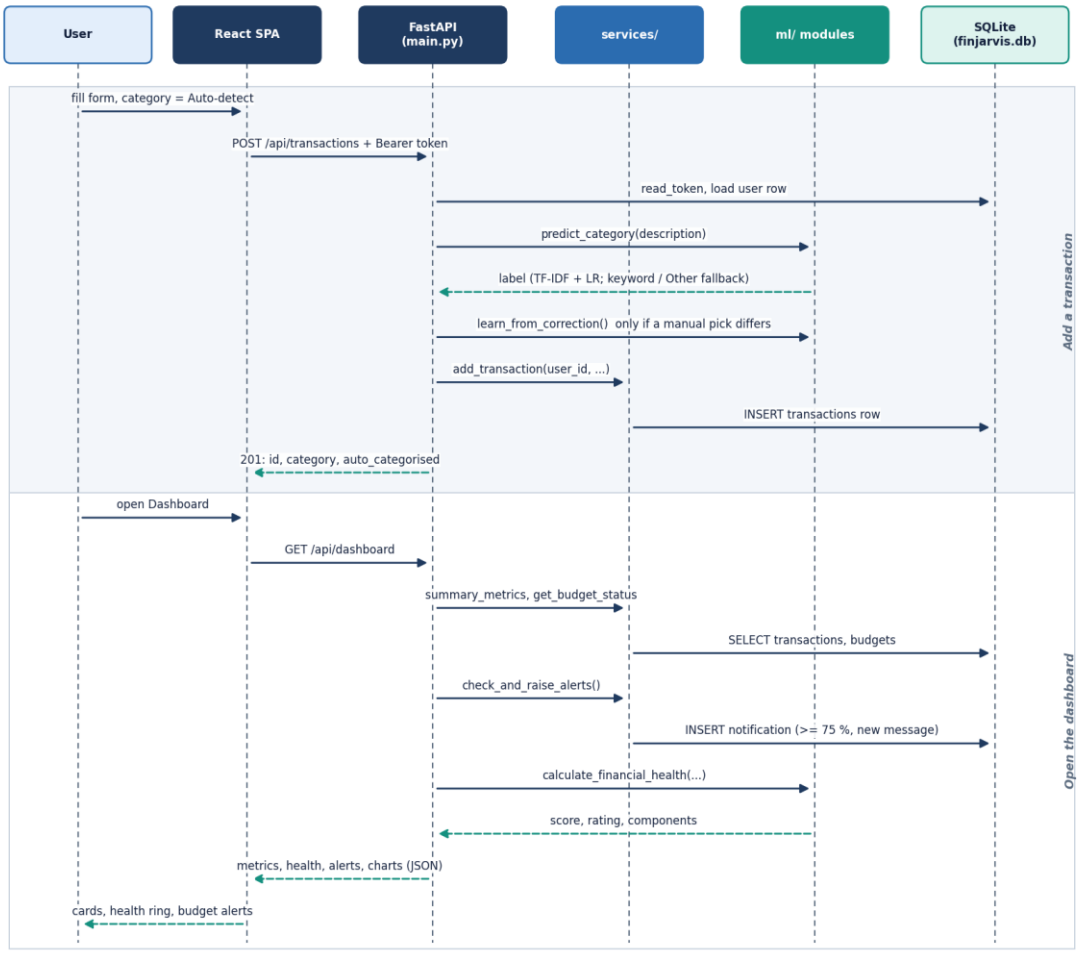


Figure 7: Sequence diagram: adding a transaction with automatic categorisation, then opening the dashboard, which computes the budget status, raises alerts and scores financial health.

The browser posts the new transaction with the bearer token; the API reads the token, loads the user row and calls predict_category on the description. Only when the user picked a category that differs from the prediction does the API call learn_from_correction, which retrains and re-saves the pickled model. add_transaction then inserts the row and the API answers 201 with the id and the category. When the dashboard is opened later, the API asks the services for the summary metrics and the budget status, calls check_and_raise_alerts, which inserts a notification when a budget is at least 75 percent used and no identical message exists, and then calls

calculate_financial_health. The combined JSON reaches the React page, which draws the cards, the health ring and the budget alerts.

Use-case diagram

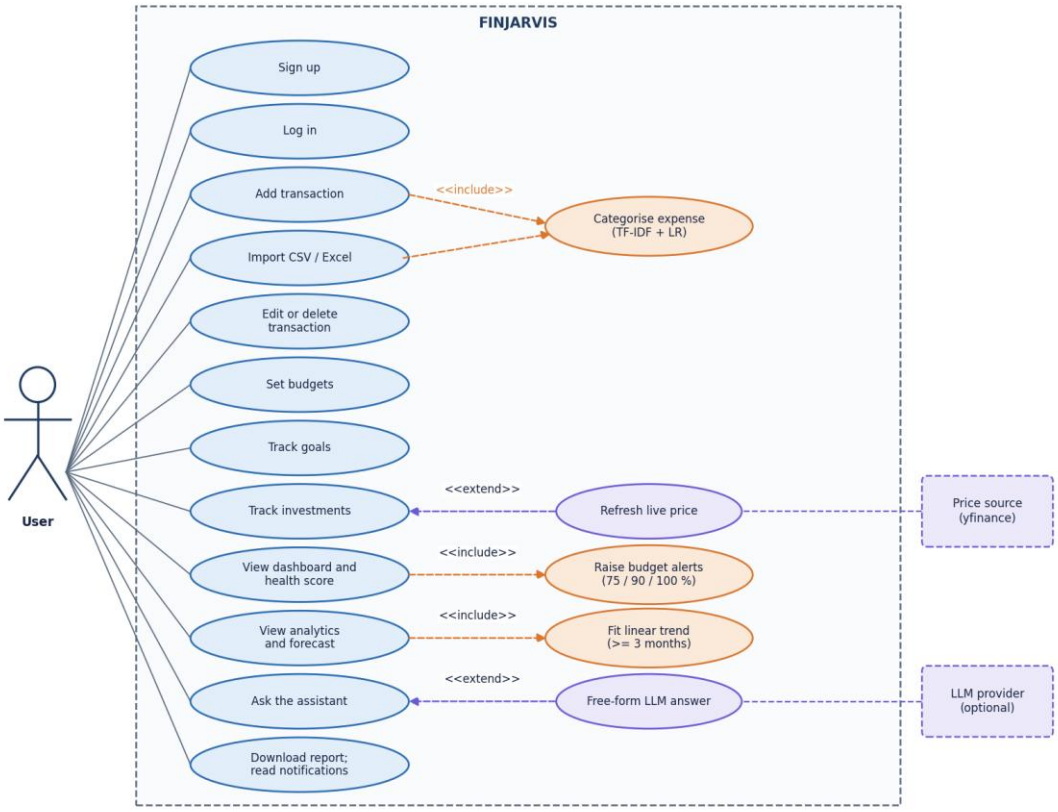


Figure 8: Use-case diagram: one actor, the signed-in user, with twelve use cases, three included behaviours and two optional extensions backed by external services.

The product has a single role: every account sees only its own data, and there is no administrator screen. The user can sign up, log in, add or import transactions, edit or delete them, set budgets, track goals and investments, view the dashboard and health score, open the analytics page and forecast, ask the assistant and download reports or read notifications. Adding and importing a transaction include the categorisation use case. Opening the dashboard includes the budget-alert check, and the analytics page includes the linear-trend forecast. Refreshing a live price extends the investment use case through the price source, and a free-form LLM answer extends the assistant use case, and both are optional.

Entity-relationship diagram

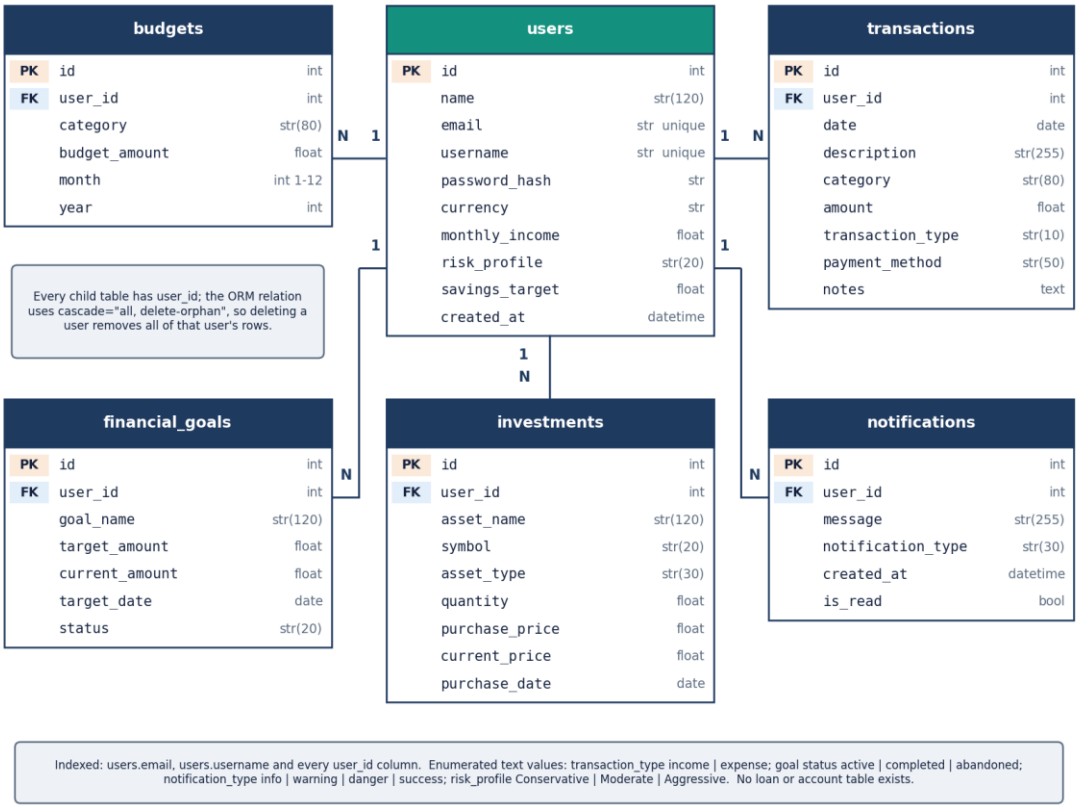


Figure 9: ER diagram of finjarvis.db: users and its five child tables (budgets, transactions, financial_goals, investments, notifications), with primary keys, foreign keys and column types.

The database has six tables. users holds the account, the password hash, the preferred currency, the monthly income, the risk profile and the savings target, and email and username are unique. transactions stores the date, description, category, amount, type (income or expense), payment method and notes; budgets holds a monthly limit per category; financial_goals holds a target amount, the amount saved so far, a target date and a status; investments holds the asset, quantity, purchase and current price; and notifications holds the alert message, its type and a read flag. Each child table has a user_id foreign key and an index on it, and deleting a user removes all of that user's rows through the ORM cascade. There is no loan or account table, so loan tracking from the abstract is not part of the database.

Flowchart of the core algorithm

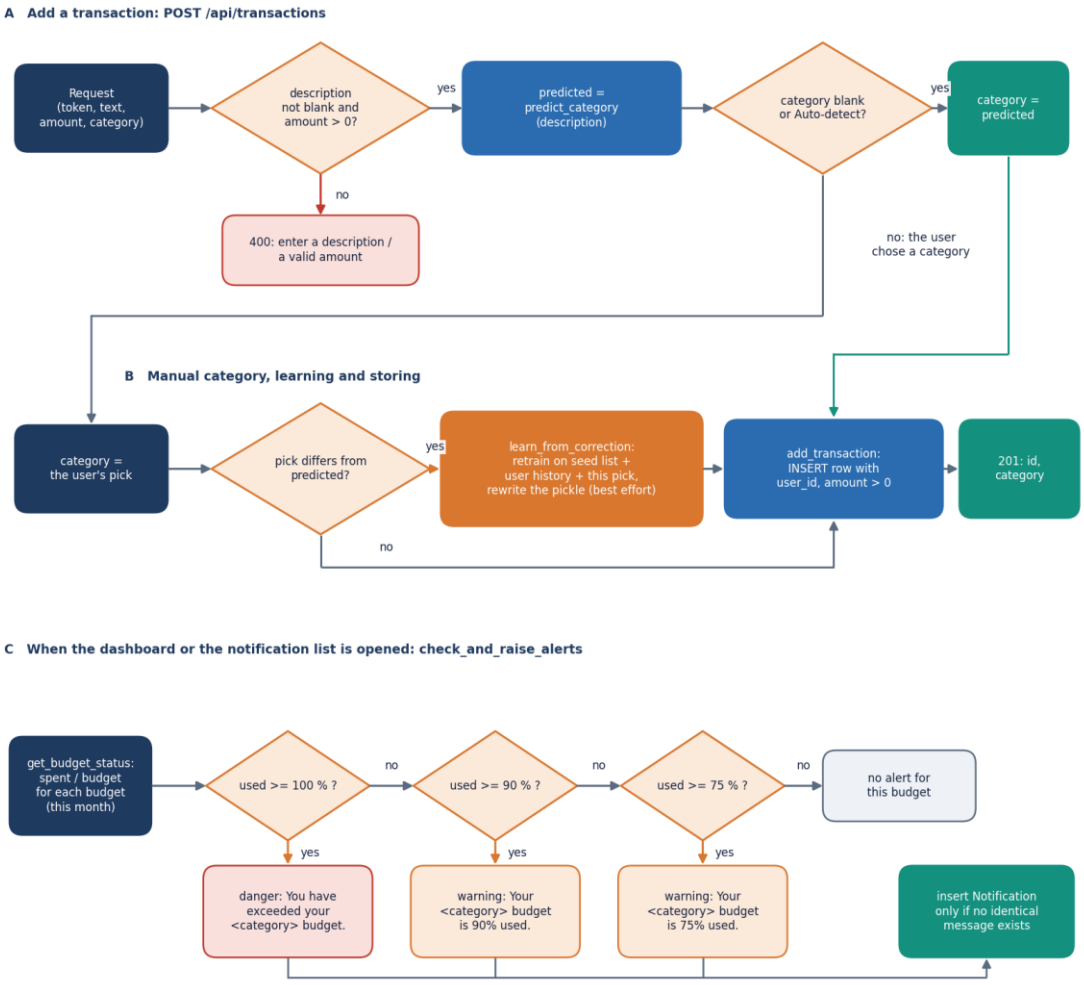


Figure 10: Flowchart of the core algorithm: validate and categorise a new transaction, store it, learn from a manual correction, and raise budget alerts at 75, 90 and 100 percent when the dashboard or the notification list is opened.

Lane A validates the request (a non-blank description and an amount above zero, otherwise 400) and asks the model for a category; if the category is blank or Auto-detect the predicted label is used. Lane B handles a manual pick: when it differs from the prediction, learn_from_correction retrains on the seed list, the user's history and the pick, and any failure there is ignored so that it never blocks the save; add_transaction then inserts the row and the API returns 201. Lane C runs when the dashboard or the notification list is opened: for each budget of the current month the percentage used is tested against 100, 90 and 75 percent in that order, and the matching message is inserted as a notification only if an identical message does not already exist. Because duplicates

are detected by message text alone, a budget that crosses the same threshold again in a later month raises no new alert.

Expense categoriser pipeline

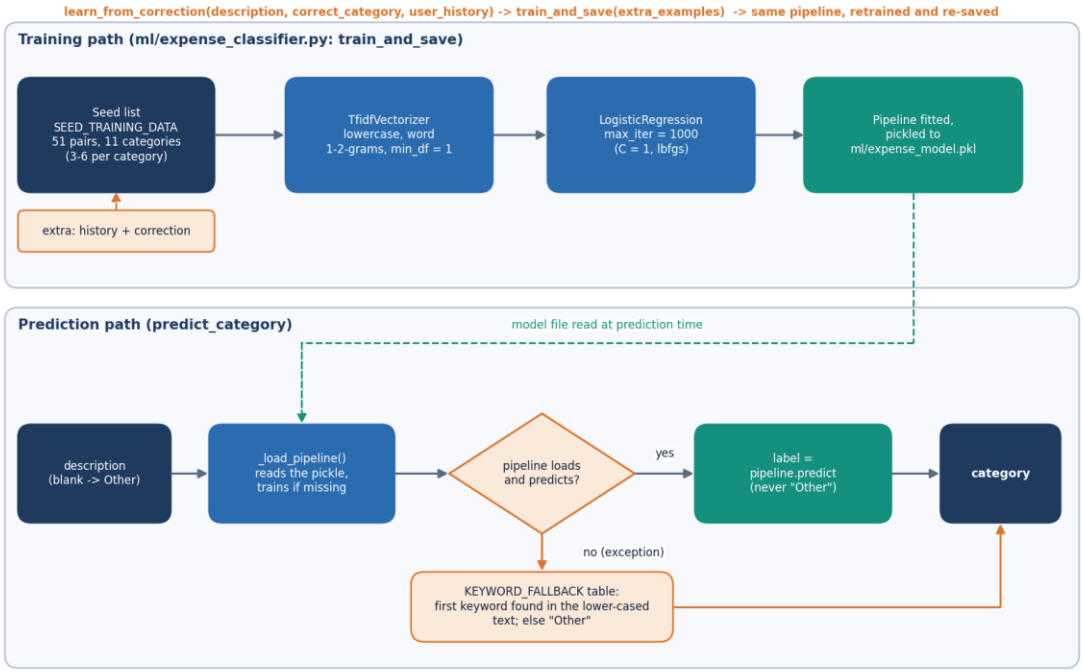


Figure 11: Training and prediction paths of the expense categoriser: seed list, TF-IDF word 1-2-grams, logistic regression, pickled pipeline, keyword fallback and retraining on a correction.

Training takes the 51 seed descriptions of 11 categories, optionally plus the user's history and a correction, fits TfidfVectorizer (lower-case, word unigrams and bigrams, min_df of 1) and LogisticRegression (max_iter 1000), and pickles the fitted pipeline to ml/expense_model.pkl. Prediction returns Other for a blank description, otherwise loads the pickle (training it first if it is missing) and returns pipeline.predict for the text. If loading or predicting raises an error, the first keyword of a fixed table found in the lower-cased text decides the category, and Other is the last resort. As long as the model loads it never outputs Other, and text made only of unseen words produces a label from the model's intercepts, which is the weakness measured in the results chapter. learn_from_correction simply calls the same training routine with the extra examples.

Financial health score model

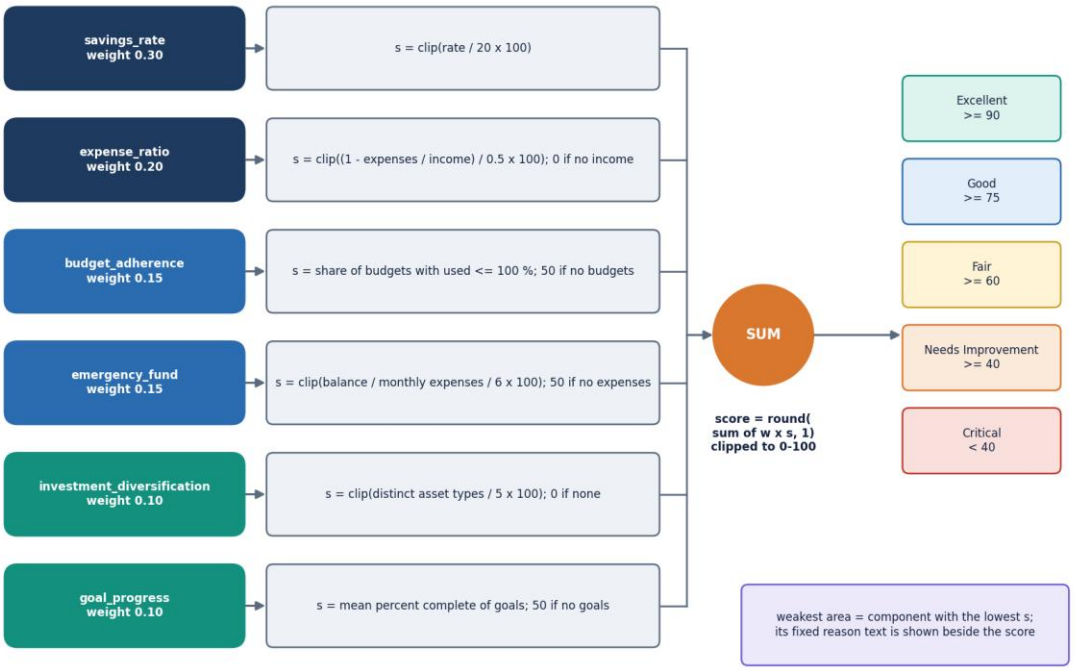


Figure 12: Financial health score: six component scores with weights 0.30, 0.20, 0.15, 0.15, 0.10 and 0.10 are summed into a 0-100 score and mapped to five rating bands.

Each component is first clipped to the range 0 to 100: the savings rate counts fully at 20 percent of income, the expense ratio is full at half of income or less and zero when expenses equal income, budget adherence is the share of budgets at or below 100 percent, the emergency fund is full at six months of expenses covered by the balance, diversification is full at five distinct asset types, and goal progress is the mean percent complete. Components with no data use fixed neutral values (50 for budgets, goals and the emergency fund without expenses, 0 for diversification and for the expense ratio without income). The weighted sum is rounded to one decimal and mapped to Excellent (90 and above), Good (75), Fair (60), Needs Improvement (40) or Critical, and the lowest component is reported as the weakest area with a fixed explanatory sentence.